

“Semaphore” 之二



Semaphore组合Condition对象

北京理工大学计算机学院
金旭亮

使用Semaphore实现生产者消费者模式



由于Semaphore允许一次有多个线程投入运行，因此，可以用它来实现一种复杂的生产者消费者模式：

多个生产者-多个消费者

但要实现这种复杂的场景，仅仅使用Semaphore还不够，还必须引入一种新的机制——Condition，为此，在介绍生产者-消费者实例之前，先来说说Condition。

Condition



使用Semaphore仅能保证一次只有指定数目的线程访问共享资源并执行特定的读写操作，但无法通知其它线程—某事情发生了，因此，难以精确地控制多个读写线程的推进顺序和特定场景下能执行的特定操作。

为了解决这个问题，可以引入“**条件变量 (Condition Variable)**”，它是一个实现了Condition接口的对象。

Condition

 	await()	void
 	await(long, TimeUnit)	boolean
 	awaitNanos(long)	long
 	awaitUninterruptibly()	void
 	awaitUntil(Date)	boolean
 	signal()	void
 	signalAll()	void

Condition是一个接口。使用它所定义的await()方法以判断在当前条件下是否能获得锁的使用权。如果返回值为false，则线程将挂起直到其他线程调用Signal()或signalAll()方法把其唤醒。

Condition对象的实例化



要实例化一个Condition对象，需要使用ReentrantLock组件：

```
ReentrantLock myLock = new ReentrantLock();
```

```
Condition myCondition = lock.newCondition();
```



有了Condition对象引用之后，就可以用它来同步线程了。

使用锁与Condition实现线程同步



Condition方法的调用，必须放到临界区中执行：

```
// 申请获取锁，如果成功申请到了锁，则执行后面的代码
myLock.lock();

while(某条件不满足时) {
    // 放弃当前锁，等待其他线程干完某件事情
    // 直到其它线程调用了myCondition.signal()方法，循环才退出
    myCondition.await();
}

// 干自己的事

mylock.unlock();
```

在此“暂停”

发送通知的线程



某线程获得锁之后，干某件事，干完之后通知其他等待此事干完的线程：

```
myLock.lock();
```

```
//干某件事
```

```
//通知其他等待的线程，某事已经干完
```

```
myCondition.signal();
```

```
Mylock.unlock();
```

此时，以前调用await()方法的线程
将被“唤醒”，继续工作.....



注意：调用await和signal方法的Condition对象，必须是同一个实例。

简单小结一下



首先创建`ReentrantLock`对象，再创建`Condition`对象。



线程调用`Condition`对象的`await()`方法，将会阻塞等待。



另一个线程调用`Condition`对象的`signal()/signalAll()`方法，会让阻塞等待的线程投入运行。



`await/signal`这两者是配对使用的。

Condition vs. wait/notify

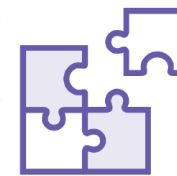


Condition接口定义的await方法、signal方法和signalAll方法分别相当于Object.wait()、Object.notify()和 Object.notifyAll()。

与Object.wait/notify的实现原理类似，每个Condition实例内部都维护了一个用于存储等待线程的队列(等待队列)，await时，当前线程入队，signal/signalAll时，线程出队。



Condition实际上就是将原先“比较粗粒度”的锁定，切分为了更为“细粒度”的锁定，以便能更为精细地控制相关线程的协作推进顺序。更通俗一点地说，使用Condition，能让相关的线程各自等待不同的事件发生。



好，下面我们可以看看如何组合使用
Semaphore与**Condition**实现多生产者多消费
者的多线程设计模式了.....

多生产者多消费者模式示例



```
class SharedResource {  
    //5个生产者  
    private Semaphore writerSemaphore = new Semaphore( permits: 5);  
    //3个消费者  
    private Semaphore readerSemapnore = new Semaphore( permits: 3);  
    //用于实现互斥读写  
    private ReentrantLock lock = new ReentrantLock();  
    //同步生产者线程的写入操作  
    private Condition addCondition = lock.newCondition();  
    //同步消费者线程的读取操作  
    private Condition removeCondition = lock.newCondition();  
  
    //最多可以放4个字符串  
    String[] store = new String[4];  
}
```

每个线程代表
一个消费者或
生产者

两个Conditon
实例，实现读
写协作。

这个实际上就是仓库。

判断资源的可用状态



```
//只要有一个位置为null,就表示可以写入数据
private boolean isEmpty() {
    for (String s : store) {
        if (s != null) {
            return false;
        }
    }
    return true;
}
```

```
//全部位置都不为null,表示没有位置可以写入
private boolean isFull() {
    for (String s : store) {
        if (s == null) {
            return false;
        }
    }
    return true;
}
```



这两个函数实际上就是检测“仓库”是否有空位让生产者放入产品。

数据的写入

addCondition: 用于等待“有数据读”的通知。

removeCondition: 用于等待“数据已取走”的通知。

```
//供生产者线程调用，写入数据
1 usage
public void addString(String str) throws InterruptedException {
    try {
        writerSemaphore.acquire(); //尝试获取写数据许可
        lock.lock(); //定义临界区，实现互斥访问
        while (isFull()) {
            //仓库满，没有位置可写，等待“有空位”的通知
            addCondition.await();
        }
        //有空位了，找到空出来的位置，写入数据
        for (int i = 0; i < store.length; i++) {
            if (store[i] == null) {
                store[i] = str;
                System.out.println(str + " 已放入第" + i + "个位置");
                break;
            }
        }
        removeCondition.signalAll(); //通知读线程：“你们可以读了.....”
    } finally {
        lock.unlock();
        writerSemaphore.release();
    }
}
```



数据的读取



```
//供消费者线程调用，读取之后清除数据
1 usage
public void removeString() throws InterruptedException {
    try {
        readerSemaphore.acquire(); //申请获得读许可
        lock.lock(); //定义临界区，实现互斥访问
        while (isEmpty()) {
            //如果没有数据，则等待"有货可取"的通知
            removeCondition.await();
        }
        //找到有数据的地方，读出来，再将其置为null，现在生产者线程可以在此位置写入
        for (int i = 0; i < store.length; i++) {
            if (store[i] != null) {
                System.out.println(Thread.currentThread().getName() +
                    "移出第" + i + "个位置的字符串: " + store[i]);
                store[i] = null;
                break;
            }
        }
        //通知生产者线程："东西我拿走了，现在你可以写了....."
        addCondition.signalAll();
    } finally {
        lock.unlock();
        readerSemaphore.release();
    }
}
```

测试代码



初始化

```
public class ProducerAndConsumer {  
    //共享资源  
    static SharedResource sharedResource = new SharedResource();  
    //用于统计消费者总共读取的数据条数  
    static AtomicInteger readCount = new AtomicInteger();  
  
    public static void main(String[] args) throws InterruptedException {  
        readCount.set(0);  
        Scanner scanner = new Scanner(System.in);  
        System.out.println("敲回车创建生产者线程");  
        scanner.nextLine();  
    }  
}
```

生产情况



```
//创建8个生产者线程，每个线程生产5个字符串，一共生产40个字符串
for (int i = 0; i < 8; i++) {
    Thread writeThread = new Thread(() -> {
        var threadName : String = Thread.currentThread().getName();
        try {
            for (int j = 0; j < 5; j++)
                sharedResource.addString(threadName + "写入数据" + j);
            TimeUnit.SECONDS.sleep(timeout: 2);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }, name: "生产者" + i);
    writeThread.setDaemon(true);
    writeThread.start();
}
```

消费情况



```
System.out.println("敲回车创建消费者线程");
scanner.nextLine();
//5个消费者
for (int i = 0; i < 5; i++) {
    Thread readThread = new Thread(() -> {
        var threadName :String = Thread.currentThread().getName();
        try {
            //只要还有数据, 就不断地消费
            while (true) {
                sharedResource.removeString();
                //统计已消费的数据数量
                readCount.addAndGet( delta: 1);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }, name: "消费者" + i);
    readThread.setDaemon(true);
    readThread.start();
}
scanner.nextLine();
System.out.println("主线程退出。");
System.out.println("一共消费了" + readCount.get() + "条数据");
```

用于避免主线程退出
过早

运行截图



```
Run: ProducerAndConsumer x
消费者3移出第1个位置的字符串: 生产者0写入数据2
消费者0移出第2个位置的字符串: 生产者0写入数据3
消费者2移出第3个位置的字符串: 生产者0写入数据4
生产者5写入数据1 已放入第0个位置
消费者4移出第0个位置的字符串: 生产者5写入数据1
生产者5写入数据2 已放入第0个位置
消费者4移出第0个位置的字符串: 生产者5写入数据2
生产者5写入数据3 已放入第0个位置
消费者4移出第0个位置的字符串: 生产者5写入数据3
生产者5写入数据4 已放入第0个位置
消费者4移出第0个位置的字符串: 生产者5写入数据4

主线程退出。
一共消费了40条数据

Process finished with exit code 0
```

小结



Semaphore一次可以允许多个线程投入运行，这是它的最大特点。



Semaphore配合Condition，可用于实现“多生产者” - “多消费者”设计模式。

编程挑战



假设某台打印服务器，可以管理3台打印机。待打印的文档排成一个队列，顺序打印。

文档可以在3台打印机中空闲的任意一台上打印。

请编写一个示例程序，使用Semaphore管理这三台打印机，模拟多个文档顺序使用这三台打印机打印的场景。

参考解答：printqueue包中的示例代码